

MiniChess AI – Design, Search, and a Neural-Network Evaluation Study

Mini Project 2

Student ID: 114000110

June 18, 2026

Abstract

This report documents a strong AI for the 6×5 “MiniChess” variant. The engine combines a classical alpha-beta / Principal Variation Search (PVS) with a full modern search stack (transposition table, killer/counter/history move ordering, iterative deepening, aspiration windows, null-move pruning, late-move reductions, futility pruning, quiescence search) and a *hybrid evaluation* that blends a hand-crafted heuristic with a small *NNUE* (Efficiently Updatable Neural Network) trained from scratch on self-play data. Beyond the working engine, the report presents an empirical study: porting Stockfish’s NNUE architecture ideas (SFNNv1–v14) to MiniChess, int16 quantization, texel tuning, and a search-allocation optimization. The central, repeatedly-confirmed finding is that **on a tiny board, search depth dominates evaluation sophistication**, and that **training-loss improvements do not imply playing-strength improvements**. The final engine beats every provided baseline (weak/strong minimax, alpha-beta, PVS) with both colours and the visible boss.

Contents

1	Overview and architecture	1
2	State value function (evaluation)	2
2.1	Hand-crafted evaluation	2
2.2	NNUE evaluation	2
3	Tree search	2
3.1	From minimax to negamax	2
3.2	Alpha-beta pruning	3
3.3	Principal Variation Search (PVS)	3
3.4	Quiescence search	3
3.5	Transposition table (TT)	3
3.6	Move ordering	3
3.7	Selectivity: pruning, reductions, extensions	4
3.8	Iterative deepening, aspiration windows, time management	4
4	NNUE: a neural evaluation trained from scratch	4
4.1	Feature encoding (side-to-move relative)	4
4.2	Network and training pipeline	4
4.3	Inference: sparse, per-leaf, float	5
4.4	Blend tuning	5

5	Architecture study: porting Stockfish NNUE (SFNNv1–v14)	5
5.1	What changed across SFNN versions	5
5.2	Dual-perspective accumulator (SFNNv1 core)	5
5.3	int16 quantization (auto-vectorized)	6
5.4	Conclusion of the study	6
6	Search-speed optimization: a ply-indexed node pool	6
7	Negative results (also useful at the demo)	6
8	Results and methodology	6
9	Key points to remember for the demo	7

1 Overview and architecture

The program speaks **UBGI** (a UCI-superset protocol) over stdin/stdout. The code is split into game-agnostic and game-specific layers:

- `src/ubgi/ubgi.cpp` – protocol loop and the *iterative deepening* driver (calls the search at depth 1, 2, 3, . . . , honours `go movetime`, emits `info/bestmove`).
- `src/games/minichess/state.cpp` – board representation, move generation (naive + bit-board), Zobrist hashing, terminal detection.
- `src/policy/submission.cpp` – the engine we developed: PVS search, all pruning/ordering heuristics, the hand-crafted evaluation, and the NNUE blend.
- `src/policy/nnue.hpp`, `nnue_weights.hpp` – the neural evaluator and its embedded weights.
- `tools/nnue_train.cpp` – a self-contained C++ self-play data generator and trainer (no external ML dependencies).

Rules and representation. MiniChess is chess on a 6-row $\times$ 5-column board with a *king-capture* win model: a side wins the instant it can capture the enemy king. There is no castling and pawns only promote to a queen. The board is stored as `char board[2][6][5]` (one plane per player; values 1=P, 2=R, 3=N, 4=B, 5=Q, 6=K). Draws occur by repetition and a step cap. A useful symmetry: Black’s starting array is exactly White’s rotated 180°, which the NNUE exploits (Section 4).

2 State value function (evaluation)

The evaluation returns an integer score in centipawn-like units *from the side-to-move’s perspective* (so the search is a pure negamax). The final score is a **blend** of a hand-crafted term and a neural term.

2.1 Hand-crafted evaluation

`custom.evaluate()` computes, for each side,

$$S_{\text{hand}} = (\text{material} + \text{PST} + \text{king tropism})_{\text{self}} - (\cdots)_{\text{opp}} + \text{pawn terms} + w_m \cdot \text{mobility}, \quad (1)$$

with the following components:

- **Material:** $\{P, R, N, B, Q, K\} = \{100, 500, 320, 330, 900, 10000\}$.
- **Piece-square tables (PST):** a per-piece 6×5 table adding positional value; the table is read from the side-to-move's orientation.
- **King tropism:** a bonus for own pieces being close to the enemy king (encourages attacking the king, which directly ends the game).
- **Pawn structure:** penalties for doubled and isolated pawns.
- **Passed pawns:** a bonus scaling with proximity to promotion (max near the promotion rank), added because promotion is decisive.
- **Mobility:** $w_m = 5$ times the number of legal moves available to the side to move.

Terminal positions short-circuit: a win returns $P_{\max}$, a draw returns 0.

2.2 NNUE evaluation

A small neural network (Section 4) produces a score from the same side-to-move perspective. It is blended with the hand-crafted score:

$$S = \frac{b \cdot S_{\text{nnue}} + (100 - b) \cdot S_{\text{hand}}}{100}, \quad b = \text{NNUE_BLEND} = 30. \quad (2)$$

NNUE is on by default and self-contained; `-DNO_NNUE` disables it and `-DNNUE_BLEND= $n$` overrides the mix. **Why a blend, and why only 30%?** Empirically (Section 4.4) pure NNUE is *weaker* than the hand-crafted eval on this game, but a light 30% admixture is meaningfully stronger than either component alone: the net captures outcome-correlated patterns the heuristic misses, while the heuristic supplies tactical robustness and is cheap to compute.

3 Tree search

3.1 From minimax to negamax

Minimax assumes the opponent minimises our score. Because MiniChess strictly alternates turns and the evaluation is symmetric (side-to-move relative), we use the *negamax* form: $\text{value}(n) = \max_m - \text{value}(\text{child}(n, m))$.

Listing 1: Negamax with alpha-beta (concept)

```
int negamax(node, depth, alpha, beta):
    if depth == 0 or terminal: return evaluate(node) // (-> quiescence)
    for each move m:
        score = -negamax(child(m), depth-1, -beta, -alpha)
        if score >= beta: return beta // beta cutoff (fail-high)
        if score > alpha: alpha = score
    return alpha
```

3.2 Alpha-beta pruning

α is the best score the side to move is already assured of; β is the best the opponent is assured of. When $\alpha \geq \beta$ the remaining moves at a node cannot affect the result (the opponent would avoid this line), so the branch is pruned. Alpha-beta returns the *same* value as minimax but visits far fewer nodes; with good move ordering it approaches the theoretical $O(b^{d/2})$ instead of $O(b^d)$.

3.3 Principal Variation Search (PVS)

PVS assumes the first (best-ordered) move is best: it searches that move with a full window $[\alpha, \beta]$ and every later move with a *null window* $[\alpha, \alpha+1]$ (a cheap test of “is this move worse than the best so far?”). If a null-window search unexpectedly fails high, it is re-searched with the full window. This is implemented in both `minimax.cpp` (the reference) and `submission.cpp` (the tuned engine).

3.4 Quiescence search

At depth 0 the engine does not evaluate immediately; it runs a *quiescence* search over “noisy” moves (captures and promotions) until the position is quiet, preventing the *horizon effect* (mis-evaluating a position in the middle of a capture sequence). It uses a stand-pat cutoff, MVV-LVA ordering, and *delta pruning* (skip captures that cannot raise α even with a generous margin).

3.5 Transposition table (TT)

A $2^{22} \approx 4 \cdot 10^6$ -entry hash table caches search results keyed by the Zobrist hash. Each `TTEntry` (≈ 10 bytes) stores a 32-bit key check, score, depth, a bound flag (exact / lower / upper), a 16-bit packed best move (`CompactMove`), and a generation byte for ageing. The TT (i) returns a cached score when a position is reached again at sufficient depth and (ii) feeds the *TT move* to move ordering, which is the single biggest ordering win. Mate scores are stored ply-adjusted so they remain correct across transpositions.

3.6 Move ordering

Good ordering makes alpha-beta/PVS effective. Moves are tried in this priority:

1. **TT move** (the best move found previously at this position).
2. **Captures by MVV-LVA** (Most Valuable Victim – Least Valuable Attacker: $16 \cdot V_{\text{victim}} - V_{\text{attacker}}$).
3. **Promotions**.
4. **Killer moves**: two quiet moves per ply that caused a cutoff.
5. **Counter move**: the quiet move that previously refuted the opponent’s last move.
6. **History heuristic**: quiet moves scored by how often they have caused cutoffs (indexed by from/to square).

3.7 Selectivity: pruning, reductions, extensions

On top of plain alpha-beta the engine searches *selectively*:

- **Null-move pruning (NMP)**: give the opponent a free move; if the position is still $\geq \beta$ at reduced depth ($R = \min(d-1, 3+[d \geq 7])$, clamped ≥ 2), prune. Disabled in check and at PV nodes.
- **Late-move reductions (LMR)**: late, quiet, non-killer moves are searched at reduced depth (reduction 1–4 growing with move index and depth, less at PV nodes); if a reduced search beats α it is re-searched at full depth.
- **Reverse futility / static null move**: at low depth, if the static eval exceeds β by $120 \cdot d$, return early.
- **Futility pruning**: at low depth, skip quiet moves when $\text{static} + 150 \cdot d \leq \alpha$.
- **Late-move pruning (LMP)**: at very low depth, skip quiet moves beyond a move-count threshold.

- **Mate-distance pruning:** tighten the window using known mate bounds so faster mates are preferred.
- **Check extension:** search one ply deeper when in check.
- **Internal iterative deepening (IID):** at PV nodes lacking a TT move, do a shallow search first to obtain a good first move.

3.8 Iterative deepening, aspiration windows, time management

The driver searches depth $1, 2, \dots$, reusing the previous depth’s results (TT + PV) for ordering. From the previous score it opens a narrow *aspiration window* (± 50); on a fail-high/low it widens exponentially and re-searches. Time is checked every 2048 nodes; the search aborts shortly before the per-move budget using a margin of $\min(300, \text{movetime}/25)$ ms. A “half-time” rule avoids starting a new depth that almost certainly cannot finish. **The competition limit is a strict 2 seconds per move**; the engine adapts to whatever `movetime` the runner passes.

4 NNUE: a neural evaluation trained from scratch

No pre-trained weights or ML libraries were used; the entire pipeline is in C++.

4.1 Feature encoding (side-to-move relative)

The board is canonicalised to the side to move so the same weights serve both colours. White keeps orientation; **Black is rotated** 180° ($r \rightarrow 5-r$, $c \rightarrow 4-c$), which is exact because Black’s start array is White’s rotated 180° . Each occupied square contributes one binary feature:

$$\text{feature} = \underbrace{\text{base}}_{\substack{0 \text{ own, } 180 \text{ enemy}}} + (\text{piece} - 1) \cdot 30 + \text{sq}, \quad \text{sq} = \begin{cases} r \cdot 5 + c & \text{White} \\ (5-r) \cdot 5 + (4-c) & \text{Black} \end{cases} \quad (3)$$

giving $2 \times 6 \times 30 = 360$ features (own/enemy $\times$ 6 piece types $\times$ 30 squares). The encoding is implemented identically in the trainer and the engine – a mismatch would silently corrupt the evaluation.

4.2 Network and training pipeline

The shipped network is a small fully-connected net $360 \rightarrow 32 \rightarrow 1$ with ReLU and a tanh output (scaled by 600 to centipawns). Training (`nnue_train.cpp`):

1. **Self-play data:** 3000 games are played (random openings of 1–8 plies plus 12% random moves for diversity, otherwise a depth-4 search picks moves), yielding $\approx 98\text{k}$ positions.
2. **Labels (WDL):** every position is labelled with the eventual game result from *its* side-to-move’s perspective (+1 win, 0 draw, -1 loss). Outcome labels carry information beyond the current eval, which is the whole point.
3. **Training:** Adam, mini-batch MSE against the tanh output, 60 epochs (final MSE ≈ 0.56).
4. **Embedding:** weights are written to `nnue_weights.hpp` as plain C++ arrays, so the engine is self-contained (no runtime model file) and compiles with the standard command.

4.3 Inference: sparse, per-leaf, float

The board has ≤ 20 pieces, so the accumulator is rebuilt *sparingly from scratch* at every leaf – summing the ≤ 20 active feature rows of W_1 – rather than incrementally. This avoids make/unmake

accumulator bugs and is cheap enough that NPS stays high (~ 1.4 M nodes/s). Output:

$$\text{cp} = \text{round}\left(\tanh\left(b_2 + \sum_h W_2[h] \cdot \text{relu}(b_1[h] + \sum_{f \text{ active}} W_1[f][h])\right) \cdot 600\right). \quad (4)$$

4.4 Blend tuning

We measured several blend ratios with *diverse openings* (see Section 8). Pure NNUE ($b=100$) lost to a $b=50$ blend 0/8; higher NNUE weight was monotonically worse ($b=70$ scored 25% vs $b=50$). The chosen $b=30$ beats the pure hand-crafted engine **62.5%** and the boss **87.5%** (diverse openings), and sweeps the boss **8/8** from the fixed start at 2s.

5 Architecture study: porting Stockfish NNUE (SFNNv1–v14)

We investigated whether Stockfish’s NNUE architecture evolution transfers to MiniChess. This section is the most likely demo discussion point.

5.1 What changed across SFNN versions

Version	Key architectural idea	Transfers to MiniChess?
SFNNv1	HalfKP features; <i>two-perspective</i> accumulator; clipped ReLU; deeper head; int quantization	dual-persp / clipped-ReLU: yes; HalfKP: too many features
SFNNv4	HalfKAv2 (king included as a piece)	partial (we include the king)
SFNNv5	HalfKAv2_hm (horizontal mirroring)	no – board is not L/R symmetric
SFNNv6	L1 width doubled; <i>piece-count output buckets</i>	buckets: cheap, plausible
v7–v14	L1 scaling 1024 $\rightarrow$ 3072, more data	scaling only

5.2 Dual-perspective accumulator (SFNNv1 core)

We implemented the defining NNUE idea: encode the position from *both* the side-to-move’s and the opponent’s viewpoint through a *shared* feature transformer, concatenate the two accumulators, and pass through a deeper head ($360 \rightarrow 64[\times 2] \rightarrow 32 \rightarrow 1$) with clipped ReLU. Trained on pure WDL it reached a much lower MSE (0.237 vs 0.56) – yet it **played worse**: the two accumulators plus the bigger head cut NPS $\sim 3\times$ ($1.26\text{M} \rightarrow 350\text{k}$), costing $\sim 1.5\text{--}2$ plies of depth. Result: 25% vs hand-crafted, 80% vs boss (down from 62.5%/87.5%).

5.3 int16 quantization (auto-vectorized)

To recover the lost speed we quantized to int16 (SFNNv1’s speed trick): weights scaled by $Q_A=512$ (clipped-ReLU unit) and $Q_W=256$ (layer weights), with *int32* accumulation laid out contiguously so the compiler auto-vectorizes under `-march=native` (float-vs-int parity ≈ 2.5 cp). This raised the dual-perspective net to 502 knps and 45%/85% – a clear recovery, but *still below* the simple single-perspective net. Quantizing the single-perspective net instead gave *no* speedup (its float eval is already cheap and int16 widening cancels the lane-count gain) and the clipped-ReLU retraining slightly regressed it (47.5% vs hand).

5.4 Conclusion of the study

Stockfish’s NNUE architecture is sound but does not transfer to MiniChess. On full chess, NNUE’s eval cost is amortised by enormous search trees, the hand-crafted alternative is far weaker, and int8 SIMD + incremental updates keep NNUE cheap. On a 6×5 board none of that holds: a cheap eval plus more search depth wins, so the structural sophistication (and even its quantization speed trick) is a net loss. The shipped engine therefore uses the *single-perspective float* net. (All experiments are preserved on git branches as documented negative results.)

6 Search-speed optimization: a ply-indexed node pool

Profiling pointed at per-node memory churn: the search did `new State+delete` (and a fresh move-list allocation) for every node. We added `State::apply_into / null_into`, which write the child position *in place* into a caller-owned `State` taken from a ply-indexed pool (`g_node_pool`), reusing each slot’s move-list buffer. Depth-first search guarantees `pool[child_ply]` is free to overwrite once a subtree returns, and a parent never reads a child slot while recursing – so the search tree is *provably identical* (verified: same node counts and scores) while two heap allocations per node disappear. Net effect: **~12% higher NPS** ($1.26\text{M} \rightarrow 1.41\text{M}$), validated at 10/12 vs the boss. This is the one optimization (of the three attempted) that shipped.

7 Negative results (also useful at the demo)

- **Texel-tuning the material values** to fit WDL improved the static loss by only 0.4% and *lost 37.5%* head-to-head: static-eval loss is a poor proxy for strength, and material values also drive move ordering / delta pruning, so reweighting hurt search efficiency.
- **Opening book:** with a strict *per-move* 2s limit (no game clock) a book cannot bank time and the engine already plays the fixed opening optimally, so a book would only echo the engine’s own move. Not shipped (it would be a no-op).
- **Search-score teacher for NNUE** (labelling positions with the engine’s own deep-search score instead of game outcome) reached a very low MSE but regressed in play – the net merely learned to imitate the existing eval.

8 Results and methodology

Measuring strength correctly. Games from the fixed start position are nearly deterministic (only timing jitter varies), so a “20-game match” is effectively one game repeated and cannot resolve small differences. We therefore added *random openings* (`--opening-plies N --seed S`) to the match runner; all comparative numbers below use diverse openings.

Constraints respected. Standard library only (NNUE weights are embedded C++ arrays); no manual SIMD/inline-asm (only compiler auto-vectorization via `-march=native`); the engine’s policy code is single-threaded; TT ≈ 48 MB, well under the 4 GB limit.

9 Key points to remember for the demo

1. **Search:** negamax + alpha-beta + PVS + quiescence, plus TT, killer/counter/history ordering, IID, aspiration windows, NMP, LMR, reverse-futility/futility/late-move pruning, mate-

Opponent	Result (final engine)	Notes
Weak MiniMax (baseline)	4/4	both colours
Strong MiniMax (baseline)	4/4	both colours
Alpha-Beta (baseline)	win	weaker than the PVS boss we beat 8/8
PVS (visible boss)	8/8	2s, fixed start
PVS boss, diverse openings	87.5%	20 games, harder test
Hand-crafted-only engine	62.5%	isolates the NNUE blend's value

Table 1: Final engine (single-perspective NNUE blend, node pool) vs. opponents.

distance pruning, and check extensions. Be ready to explain *why* each prunes safely.

2. **Evaluation:** material + PST + king tropism + pawn structure + passed pawns + mobility, *blended* with a from-scratch NNUE at 30%.
3. **NNUE:** 360-feature side-to-move-relative encoding (180° flip for Black), $360 \rightarrow 32 \rightarrow 1$, trained by C++ self-play on WDL labels, weights embedded in a header, sparse per-leaf inference.
4. **Headline insight:** *on a small board, depth beats eval sophistication*; SOTA chess NNUE architecture and its quantization trick do *not* transfer (measured, not assumed).
5. **Second insight:** *lower training loss $\neq$ stronger play* – shown three times (score-teacher NNUE, dual-perspective NNUE, texel tuning).
6. **Methodology:** use random openings to measure strength; a clean speedup (node pool) must be a provable non-regression (identical node counts) before shipping.

Build.

```
g++ --std=c++2a -Wall -Wextra -Wpedantic -g -O3 -march=native -static \
-Isrc/games/minichess -Isrc/state -Isrc \
-o build/minichess-ubgi.exe \
src/games/minichess/state.cpp src/policy/minimax.cpp \
src/policy/random.cpp src/policy/submission.cpp src/ubgi/ubgi.cpp
```

NNUE is on by default and self-contained; `-DNO_NNUE` disables it and `-DNNUE_BLEND=n` changes the mix.